



ROBOCUPJUNIOR RESCUE MAZE 2026

TEAM DESCRIPTION PAPER

Gebruder Weiss Cro Team

Abstract

- We are Gebruder Weiss Cro Team consisting of 2 members. Our robot for the competition has been developed through the last 2 years. From the last year World RCJ to this year's, we increased the precision of our robot movement and decreased the number of victim misidentifications. We also developed a system so that the robot can continue moving after LOP.
- Apart from sensors, our robot was constructed based on the design of our robot from previous years, but also having in mind observed limitations and problems that existed. We designed both 3D printed parts (including Rescue kits), as well as the PCB, which we have been perfecting over the past 3 years.
- The most innovative components of our robot include the kit dispenser, wheels that we designed for the movement over speedbumps, usage of both IR and colour sensor for the coloured tiles detection, communication between teensy and RPi 5, the usage of RPi cameras in contrast to OpenMV used by most other teams and in connected with this, the usage of a combined system using 2 algorithms for victim detection.

1. Introduction

a. Team

- *David Pongrac*, team leader and main software developer. He's responsible for maze navigation and mapping, as well as victim detection. He helped in some hardware development like the design of a PCB. He has been involved in robotics for 12 years and was a member of the team that won 2016 RCJ Portugal Open in Rescue Line Preliminary. He also won several RCJ Croatia national competition in entry categories of both Rescue Line and Maze. In 2024, he competed in European RCJ in Hannover and in 2025 in World RCJ in Salvador. He continued to develop the robot for this year's World RCJ.
- *Gregor Klaric*, responsible for hardware development. He designed all 3D printed parts as well as the PCB and was the main assembler of the robot, both mechanical and electronical parts. He participated in European RCJ in Portugal. He briefly left Maze category and participated in European RCJ in Bari in 2025 where he won 2nd place before returning this year to Rescue Maze category.

2. Project Planning

a. Overall Project Plan

Team's objective for the competition

The most important function of our robot, as the name suggests, is to be able to navigate inside a maze and to rescue victims – to get to them and give them required assistance based on the severity of the situation, while avoiding intrusions. Its secondary objective is to save itself, to be able to return back after saving victims.

The rules of the category state that the robot has 8 minutes to explore as much of the maze it can, to avoid all solve all possible challenges, such as stopping on blue tiles that slow him, avoid black tiles, going up and down ramp and stairs etc. Majority of the points come from saving victims, recognizing their type (representing severity of their situation) and the dropping (according number) of rescue kits. Lastly, if it's able to, to get back to the start tile, get exit bonus, thus giving extra points to the teams that have accomplished this task.

- **Milestones and timeline:**

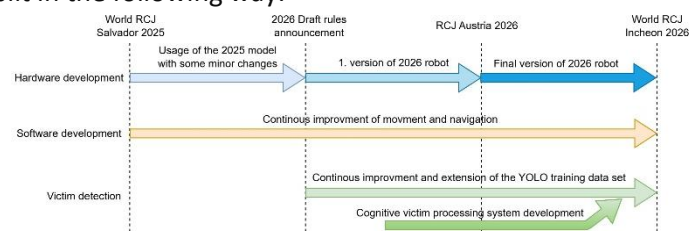
Milestone	Deadline	Member assigned	Short Description
<u>Improved movement</u>	end of October	David	Since we had several problems with our last year's movement, we made several changes in the way in which the robot moves. We tested it on the old robot.
<u>New compass integration</u>	end of November	Gregor, David	Because the problems caused by our last compass (such as losing precision), we decide to test a new one
<u>New PCB design</u>	end of November	Gregor	Since we made some electronic changes, such as adding a new compass, we needed to redesign the PCB
<u>1. complete robot prototype</u>	1. December	Gregor	Our first prototype to start incorporating navigation and motion.
<u>First YOLO model</u>	20. February	David	After learning about potential changes in rules regarding victims, we made our first victim detection model.
<u>Redesigned robot prototype</u>	end of march	Gregor	After seeing changes in rules, such as reduces amount of rescue kits, we had to make several changes.
<u>Redesigned YOLO model</u>	28. March	David	Correction mistakes in our last YOLO model.
<u>First cognitive victim detection model</u>	end of April	David	After making a very high-quality YOLO model, we focused on solving cognitive victim
<u>Improved robot model</u>	10. March	Gregor	After participating in Austria Open 2026, several problems have been seen and robot design had to be changed.
<u>Final cognitive victim detection model</u>	23. June	David	With the competition date approaching, we had to finalise the cognitive victim detection.

- **Schedule/plan agreements:**

- Since team members had other important tasks, like school and other competitions, it gathered together primarily during the weekends and made most of our project. School break gave us more time to meet more often and thus be more productive. Since we already had a high-quality project, we mostly had to focus on rules changes (blue tile, cognitive victims). Since compass was our biggest problem before we knew about any rule changes, it was done first. While one member was redesigning PCB in order for compass to fit in, the other one was researching how it functioned.

b. Integration Plan

- Due to the fact that our robot was based on the component design, we could independently access the development of the individual component before integrating them into a functional product. The software components were able to be developed independently before the robot was assembled.
- Essentially, the robot can be structurally split in the following way:
 - Power supply, drive system and orientation system
 - Victim detection
 - Recue kit ejection
- Software can be divided into following parts:
 - Mapping and motion planning



- Movement inside the maze
- Victim detection
- From the beginning, we decided that the communication between electronic components, depending on their capabilities and needs, will use I/O protocol for directly connected individual components to I/O teensy pins or connection through I²C busbar. The exception to this rule is the Bluetooth module used to show the status of the robot while we were testing. It is connected through the serial interface of Teensy,

We were able to develop both hardware and software simultaneously, after which we needed to consolidate and test them how they functioned together, especially before the competitions. After assembling and testing the overall functionality, all components that hadn't met the requirements were redesigned in order to be able to pass the tests they couldn't pass in the previous development phase.
- We considered that the component, either hardware or software, has met its requirement when the robot was able to solve the problem in at least 10 different arenas with many types of the said challenge.

3. Hardware

- Based on previous experience, we chose to have wheels with motors on each wheel. We also decided to have two batteries, one for the 3.3V sensor and motor, and the other for all other components at 5V (Teensy, RPI, LED lights, package ejector servo). The overall dimensions of the robot are 200×165mm, while the dimensions of the robot body itself are 170×110mm.

We decided to have the package ejector located in the middle of the robot with the possibility of ejection on both sides of the robot, and that the robot has two cameras for victim detection, one on the left and one on the right.

Most of the mechanical components were designed using Autodesk Fusion and printed on a 3D printer using PET-G plastic.
- In order to ensure the best possible communication between the electronic components of the robot, we made our own PCB, on which the Teensy 4.1 occupies the central place, which is the "brain" of our robot and to which all the electronic components of the robot are connected.

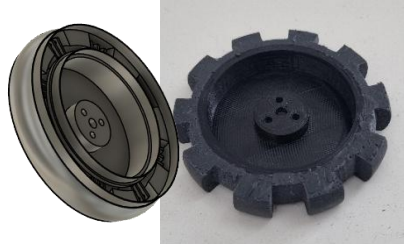
On the mechanical side, the robot has three levels, the first level is where the motors, sensors that "look" at the floor and the package ejector are held. The second level is where the cameras and part of the sensors are held, while the third level is the PCB to which the sensors and other electronics are mostly directly connected. The RPI5 is also connected to the PCB.

The RPI5 was chosen because on the one hand it has the ability to work with two cameras, while on the other hand it provides satisfactory performance in image processing and classification.

Communication between the sensors takes place either via the I²C bus or by directly connecting the sensors to the Teensy inputs.

b. Mechanical Design and Manufacturing

- **Main Structure**
 - As we wrote previously, and can be seen in the picture of the robot, the robot has three interconnected levels. All the components of the robot are attached to these levels. In order to keep the robot's center of gravity as low as possible, the batteries are placed in the space between the first and second levels.
 - The first two levels are made of 3D printed plastic, while the third level is a PCB to which all the electronic components are connected.
 - A shock absorber (suspension) is installed on the rear axle, which ensures that the robot touches the ground with all four wheels almost the entire time it moves through the maze.
- **Actuators and Power Train**

- Four z67z67u 20D 156.1 motors are used to drive the robot. The motors give the robot enough power to overcome various obstacles within the maze.
 - To make it easier for the robot to overcome obstacles on the floor, this year we made new wheels with grooves, the wheel base is made of 3D printed plastic over which silicone is poured, which ensures good grip to the surface. The picture shows the mold of our wheel into which silicone is poured, which hardens after pouring in about 24 hours.
 - The pictures show the wheel shell designed in Autodesk Fusion, and the finished wheel after pouring the silicone and removing the excess plastic that served as a mold for the silicone.
- 
- **Subassemblies/modules**
 - The design of our robot could be described as compact and modular. This means, on the one hand, that there are a large number of different components/modules, while on the other hand, these components are compactly connected to each other by means of plastic screws, nuts and spacers. Considering that all electronic components are connected to the main PCB, when building the robot, all wires need to be connected to the PCB in a good way. Disassembling the robot is of course faster than assembling it.
 - **Rescue kit deployment mechanisms**
 - When designing the ejector packer, we stuck to a few rules that we set for ourselves.
 - The packet ejector should be able to eject all packets to one side of the robot (left or right).
 - Ejecting packets should be as simple as possible and with as little chance of the packet getting stuck during ejection as possible (a problem we had in the previous version).
 - The package ejector should be placed at the same level as the camera, to ensure that the packages are ejected in the direction of the victim's mark on the wall.
 - Considering the set task when designing the ejector, there were certain challenges that were successfully overcome, and the appearance of the ejector itself can be seen in the pictures.
 - A single MG90S 180° servo motor is used to drive the ejector. All packets in the container are supported by the ejector lever. Depending on whether the packet is ejected to the left or right, the lever is moved to the right or left side and the packets fall to the bottom of the ejector, and when the lever returns to the neutral position, the packet at the bottom is ejected in the desired direction, and the ejector is ready for a new ejection.
 - Since the total number of rescue kits that can be stored was reduced (from 12 to 8), the height of a kit ejector has also been reduced.
 - **Reliability Tests and quality assurance, Manufacturing.**
 - Regardless of the fact that the assembly of the individual components looked good when designed in Autodesk Fusion, 3D printing brings challenges that need to be solved on the fly.
 - During 3D printing, PET-G leaves thin threads of material that need to be cleaned with a fine file. Also, the dimensions set during the design can be slightly changed (0.1-0.2 mm) due to the properties of the plastic, but also the printer itself, and irregularities in the 3D print are corrected using small diameter drills and files.
 - After creating the components in Autodesk Fusion and printing them on a 3D printer (a Creality Ender 3 PRO 3D printer was used for printing), the robot is built a whole.
- 

- Individual components such as the package ejector were assembled separately and their functionality was tested before being assembled into the entire robot.
- The advantages of designing and manufacturing robots using components in a modular design are that, if problems are detected in one component, it is not necessary to reprint the entire robot, but only the component that does not meet the desired functionality is changed.
- All testing of the desired functionalities of the robot was carried out during its manufacture in arenas where various obstacles were placed. If it was determined during testing that a component did not meet the set requirements or its design did not correspond to all the situations placed on the path in front of the robot, it would be corrected or completely redesigned. Of course, it was always necessary to ensure that the redesign of one component did not disrupt other components of the robot, and if necessary, corrections of multiple robot components had to be made simultaneously in order to maintain good robot performance in operation.

c. Electronic Design and Manufacturing

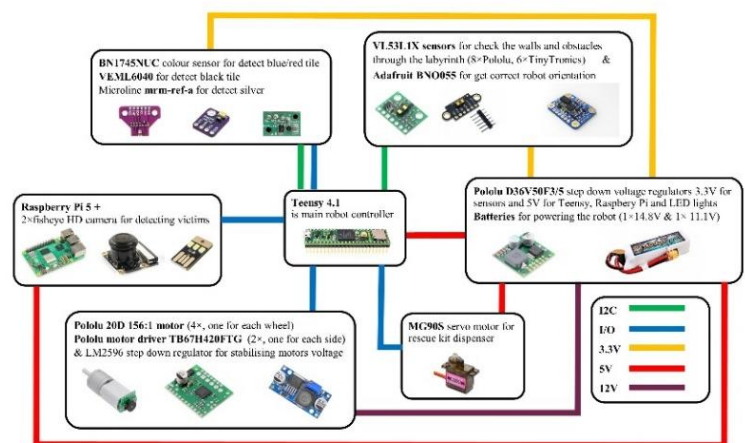
• Abstract

In the connection diagram you can see how individual components are connected to each other. All components are connected to the main PCB which forms the upper surface of the robot.

The main controller of the robot is Teensy 4.1, while RPI5 is used for image processing and searching for victims on the walls.

All communication to the main controller takes place via the I²C protocol or direct communication via the I/O pins of the controller.

Two LiPo batteries are used to power the robot, which are connected to all components via a voltage regulator.



• Sensors used

• Distance sensors

To determine the distance of the robot from walls and obstacles, 14 VL53L1X distance sensors are used. The sensors are arranged so that there are 6 sensors on the front, three sensors on each lateral side and two sensors on the back of the robot.

• Gyroscope

We use BNO085 to determine orientation in space. After the last years many problems involving this sensor, we replaced it with the new _____. However, it was performing worse than the BNO085, and after some research we found a solution for the problem with the BNO085, so we brought it back. The solution involved putting a resistor between the BNO and the battery to stabilize the voltage on the sensor.

• Floor color sensors

To determine the arrival at the blue/red field, a BN1745NUC color sensor is used, located in the middle of the robot, to minimize the influence of changes in external light on its reading. A VEML6040 color sensor and a mrm-ir-a IR sensor are used to detect the black field, because using them together we reduce the effects of outside influence like lighting. Mrm IR sensor is also used for detection silver field (check point), since the silver field can reflect colors from the environment and it can lead to inaccurate readings from the color sensor.

- **Motor counters**
On the front motors encoder sensors are installed, so that the distance traveled can be easily calculated using the number of engine revolutions. Using them the robot knows how far he has to move to go exactly one tile ahead and to calculate the height of the ramp.
- **Victim detection**
For detecting victims on walls, we use an RPI5 paired with two 160° HD cameras, using YOLO and that we trained on the data (images) we collected during our testing, as well as classic computer vision approaches for cognitive victim classification.
- **Main controller**
As we mentioned before main controller of our robot is Teensy 4.1. We decided on it because on the one hand, it has a compact design, while on the other hand, it provides enough processing power to successfully control the robot. We program the robot in C++ using Visual Studio Code with the PlatformIO plugin to transfer the program to Teensy.
- **Power subsystem**
We use two 2200mAh LiPo batteries to power the robot. Since the RPI5 requires a significant amount of energy to operate, one 14.8V battery is reserved for the 5V power supply, while the other 11.1V battery is used to power the 3.3V electronics and motors that run on 11V.
- **Custom PCB.**
We made our PCB using the Online EasyEDA tool. When drawing the wiring, we went through multiple visual checks for short circuits, and upon receipt of the finished board, we checked everything for short circuits once again using a measuring instrument.
Also, after we soldered all the passive elements (resistors, diodes,) and connectors to the board, we did another check to make sure there were no unintentional short circuits when connecting the components to the PCB.
- **Motors**
 - **Robot movement**
We use 2 Pololu's 20D 156:1 motors to drive the robot, which have proven to be reliable in previous use and deliver enough power to successfully move the robot. The only drawback we see with these motors is their relatively large size.
 - **Rescue kit ejector**
We use the MG90S 180° servo motor to operate the packer ejector, which has proven to be reliable in its task.
- **Test communication**

To make it easier for us not to have Teensy and the computer connected via cable during program testing, we installed a Bluetooth module on the robot that was set to one-way communication mode to print individual robot states to the appropriate Bluetooth interface.

4. Software

The mapping and navigation of the robot are programmed in C++ using Visual Studio Code with PlatformIO and uploaded on the Teensy which than coordinates it between sensors and motors. Program for victim detection is programmed in Python using RPi, (with YOLO algorithm as the primary victim detection mechanism).

a. General software architecture

- Our program contains 4 main structures:
 - Maze.defs** – file we connected all of our sensors to their corresponding pins so that they can get their instructions by the software system
 - Maze.h** – header file, contains every necessary constant such as tile size, encoder values in order to move exactly one tile ahead and a quarter of a circle (90°), values for black, silver, blue and red tiles, as well as every class with their attributes and functions (including getters and setters) and their data types
 - Maze.cpp** – the main part of our program. Here we program every function we defined in the header file, merge them together and give instructions when to activate them
 - VictimDetection.py** – YOLO model in combination with classic computer vision approaches is run separately on RPi 5. It's active the entire time and communicates with the Teensy by pins

Position	Maze
<pre>+ positionX: int8_t + positionY: int8_t + component: int8_t</pre>	<pre>+ allTiles[MAXIMUM_NUMBER_OF_TILES]: Tile + allTilesLastIndex: uint8_t + tilesStack[MAXIMUM_NUMBER_OF_TILES*3]: Tile* + tilesStackLastIndex: uint16_t + currentTile: *Tile + nextTile: *Tile + lastCheckpoint: *Tile + currentX: int8_t + currentY: int8_t + currentComponent: int8_t</pre>
Tile	
<pre>+ tilePosition: Position + tileType: TileType + accesibleTiles[4]: Tile* + victimType[4]: VictimType + distance: int16_t + visited: int8_t + setPosition(): void + setTileType(): void + setAccesibleTile(): void + setVictimType(): void + setDistance(): void + setVisited(): void + setUnvisited(): void + isAccesibleNeighbourTile(): bool + isSamePosition(): bool + isNextToSlope(): bool + outputTile(): void + tileTypeWeight(): int8_t</pre>	<pre>+ setCheckpointAsTile(): void + setLastCheckpoint(): void + setCurrentTile(): void + setCurrentX(): void + setCurrentY(): void + setCurrentComponent(): void + setAllDistances(): void + setDistanceOfElement(): void + addNewTile(): void + addTileToStack(): void + addVisitedTile(): void + advance(): void + setTileAsVisited(): void + mapAccesibleTile(): void + handleRamp(): void + doesUnvisitedTileExist(): bool + wallMapped(): bool + victimMappedOnMap(): Tile*: VictimType + setNextTile(): Tile* + getNextDestination(): Tile* + getOrientationToNextTile(): Tile* + shortestDistanceToNextDestination(): void + nearestNeighbour(): Tile*</pre>

Mapping:

- When mapping, the robot is positioned in the middle of the tile and uses 12 lidars to map the surrounding walls. If there is no wall, the robot knows that the 2 tiles are neighboring by making pointers to each other, and if there is a wall, it's marked as a nullptr and the robot knows that there is a wall and thus they're not neighboring. Tile type (black, blue, checkpoint, ramp...) is also mapped.

Motion and path planning:

- For these tasks, our robot functions as a **deterministic finite automaton**. It contains 12 possible states and those are:
 - SUSPEND**: The first state in which the robot is when the program is stated. Here, the robot waits until of the 2 buttons is pressed. If test button is pressed, the robot goes into TEST_HARDWARE state and if the start button is pressed, the robot goes into DECIDE_AND_SEARCH_VICTIMS state.
 - TEST_HARDWARE**: The robot enters this state when we press the test button after which it does the test procedure that depends on what are we currently testing (lidars, motors, compass etc.) and then goes back into SUSPEND state.
 - UNKNOWN STATE**: Robot enters this state when he is lost and goes immediately from into SUSPEND state.
 - DECIDE_AND_SEARCH_VICTIMS**: This may be the most important state in our automaton. In this state the robot checks wheatear the tile is blue and has to stop for 5 seconds, is it checkpoint and saves its location, wheatear its red or if it possibly a ramp tile. It also gathers information about the surrounding walls and where he needs to head next. It than uses gained information to create a Tile and save it to use it for mapping. Then he decides its next state.
 - GO_AHEAD_ONE_TILE**: When there is no wall in front of the robot, it will decide that it needs to go ahead. When there are walls on the left or the right of the robot, it is going to go ahead following them using PID algorithm and lidars to always be near the middle of the tile. When there are not walls on

either side, it will use compass to keep the direction straight. It knows how far it has to go using the number of encoder counters. If it has gone too far last time, this time he'll go less and if hasn't gone far enough, it will go more next time. While going ahead, he's checking if there are obstacles in the front or if there's a black tile, after which it goes into BLACK_TILE state.

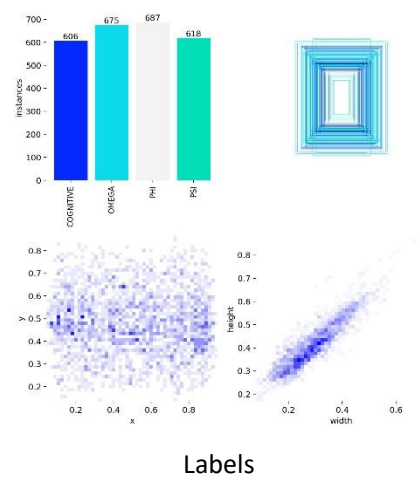
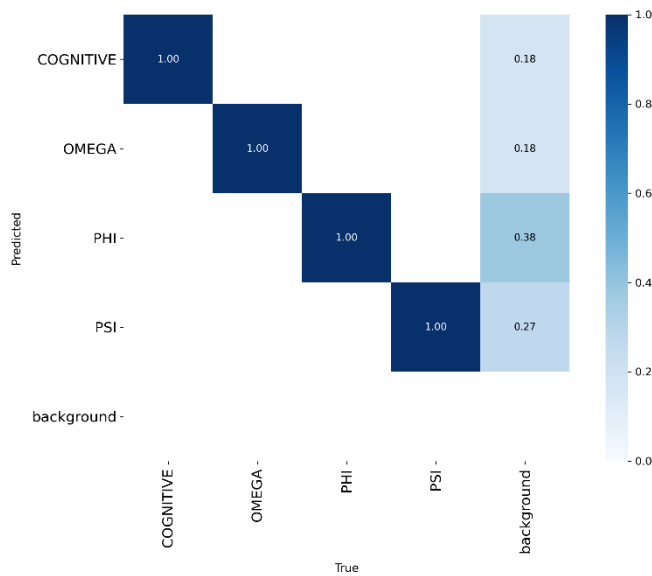
- **BLACK TILE:** If the robot detects the black tile below him, this state starts the procedure to mark that tile as a black (and it will remember that the tile is black and won't go again onto it) and to return back on the spot where she had started. We had problems that he would sometimes detect black on ramp. We solved it by adding additional condition for black detection, that the pitch difference must be very small in order for it to be declared as black.
- **TURN RIGHT:** When the next tile the robot needs to go onto is on the right or behind the current, it is going to enter into this state, turn 90 degrees right and go back into decide state.
- **TURN LEFT:** When the next tile the robot needs to go onto is on the left of the current, it is going to enter into this state, turn 90 degrees left and go back into decide state.

Diagram of robot's deterministic finite automaton

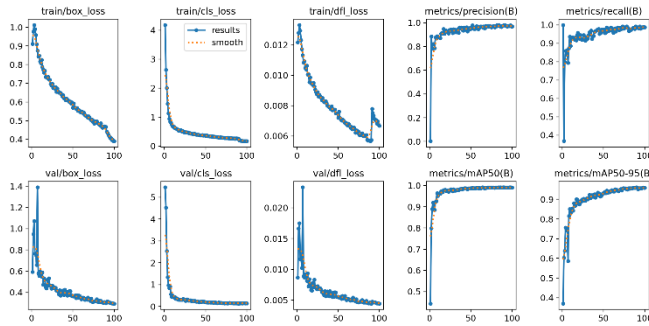


Victim detection:

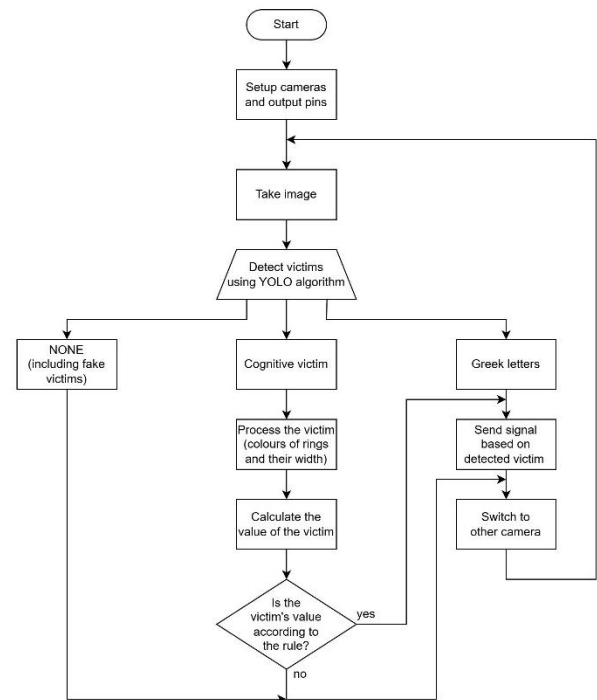
- The first layer of our victim detection is YOLOv26 algorithm. It allows our robot to know whether the victim is Ψ (Psi), Φ (Phi), Ω (Omega) or cognitive (we just detect it now to later process it). It was very easy to create our own model by training our dataset into model. We could easily improve it by adding more pictures into the model. Since the start of the development of victim detection mechanism, we knew that fake victims could appear, so we also included them into our model. Our RPI is able to work on 15FPS per side (therefore, total FPS is 30), allowing for victim detection while moving forward and when turning.
- The second layer is regarding the cognitive victims. We implemented several algorithms to process them. Using the Gaussian blur and Canny algorithm (corrected by MORPH_CLOSE function to close the potential holes) on each of the 3 image channels (R, G, B), we divide the cognitive victim into circles. After that, we use floodfill function to get the information about the number of pixels of each circle and their average colour. Using simple mathematical rules we get the information about the radius of every circle and by it the number of circles. Finally we multiply each color assigned value by the number of number of circles in each layer to get the final result about the value of the victim.
- Together, there are 6 classes of victims (Greek letters Ψ , Φ , Ω , and cognitive harmed, stable or unharmed). Each victim type sends a unique signal to the PCB using 3 pins. Those signals than get processed inside our main program, giving the robot the knowledge how many rescue kits to drop.



Normalised confusion matrix



Loss and accuracy benchmarks



Victim detection flowchart

b. Innovative solutions

- The most innovative part of our software is our approach to cognitive victim detection. Since this is a new challenge and we couldn't see how other teams solved cognitive victims. One other innovative part is the alignment after turning 90. Because the last year's turn didn't always keep him in the wanted direction, we installed this safety mechanism. The other innovative part of our program is mapping of the victims. The robot firstly checks if there is wall in the front, back or the side and if there is, it checks if a victim exists on them. If the victim is there, it's marked by a pointer. In such a way, the robot is able to know on which wall exactly the victim was and not to search again and waist rescue kits.
- Each time we made a change in our software, we would place a robot inside an arena to see how it performs. In this way, not only did we saw direct changes, but we also saw how the changes might have possible affected other parts of the program.

5. Performance evaluation

- After making each version of the robot (upgrade or other change), we began with the test of the sensors to find out if they are properly connected and then if the last version of the program is adapted to the changes because at the very least it needed to accomplish anything the previous version was able to. When developing software, we started by solving the challenges that appeared most frequently (they were usually the easing, such as non-white tiles), before moving to more complex situations like movement on ramps and avoid of obstacles. After each change, we checked how it might have impacted other parts of the program. They might occasional provide additional beneficial results like when we made avoiding obstacle, it brought a benefit of robot not bumping itself onto into the walls.
- Before we took part in a competition and concluded that we don't have enough time to finish a mayor goal, we would stop the developments and focus on speeding up the robot. The competitions also served to as tests in unfamiliar environments. They brought some new challenges that our robot might able to overcome. He would overcome some, while others wouldn't. We would replicate those he didn't overcome onto our test fields.
- **Reliability tests:**

Since one perfect test cannot guarantee the quality of a product, we would continuously place our robot trough the same test field several time. If we concluded that the robot isn't just lucky (at least 90% of tests were executed with perfection) we would then change the testing field and repeat the cycle. After concluding that a segment, either hardware or software, had been successfully done, we would make new test field.

 - **Wall mapping bug:**

Sometimes, when the robot would arrive for the first time on the unexplored tile, it would incorrectly map the existence of a wall. As a result, it wouldn't be able to return to the start and in extreme cases, it would stop in the middle of the field. To counter this, we simply added the double checking of the wall's existence.
 - **IMU inerrability:**

During the last world RCJ, our IMU experienced sudden problems, like either staring to turn uncontrollably or turning 45° . We decided to replace BNO085 with _____, but it brought a host of problems, like having a large offset of $\pm 10^{\circ}$. After finding the solution for the old BNO085, we brought it back and it was working with almost zero problems.

6. Conclusion

- To conclude, we believe that we had done an extremely good job designing, constructing and then programming our robot. We believe that this TDP gave you very detailed explanation about the function of our robot and hope that you are able to understand it. Some additional information might be provided on the poster or the video. We're sure that our rounds are going to be interesting to watch. There is almost not single situation that can appear and our robot isn't able to overcome. We invested a lot of our time, resources and energy into this project and believe that we will achieve very high results.

References

- <https://app.diagrams.net/> - WEB page where we created our flowcharts
- <https://www.visual-paradigm.com/guide/uml-unified-modeling-language/uml-class-diagram-tutorial/> - rules when making ULM diagrams

- <https://docs.ultralytics.com/guides/raspberry-pi/>
- <https://roboflow.com/> - online program where we made our YOLO models